Estatística de Ordem

Discente: Bruno Victor Paiva da Silva

1. **Implemente o algoritmo de mínimo e máximo simultâneos da seção 9.1 do livro do Cormen, 4a Ed na sua linguagem favorita e mostre através de medição de tempo que é mais rápido que a abordagem não-simultânea para um vetor de entrada suficientemente grande.**

```
#include <chrono>
#include <iostream>
#include <random>
#include <vector>
#include <algorithm>

int min_value(const std::vector<int> &a) {
    int res = a[0];
    for (size_t i = 1; i < a.size(); i++) {
        if (a[i] < res) res = a[i];
    }
    return res;
}

int max_value(const std::vector<int> &a) {
    int res = a[0];
    for (size_t i = 1; i < a.size(); i++) {
        if (a[i] > res) res = a[i];
    }
    return res;
}

void min_max(const std::vector<int> &a, int &min, int &max) {
    int n = a.size();
    int start;
```

```
    if (n % 2 == 0) {
        if (a[0] < a[1]) {
            min = a[0];
            max = a[1];
        } else {
            min = a[1];
            max = a[0];
        }
        start = 2;
    } else {
        min = a[0];
        max = a[0];
        start = 1;
    }

    for (int i = start; i < n; i += 2) {
        int local_min, local_max;

        if (a[i] < a[i + 1]) {
            local_min = a[i];
            local_max = a[i + 1];
        } else {
            local_min = a[i + 1];
            local_max = a[i];
        }

        if (local_min < min) min = local_min;
        if (local_max > max) max = local_max;
    }
}

double get_median(std::vector<double>& times) {
    std::sort(times.begin(), times.end());
    return times[times.size() / 2];
}
```

```
int main() {
    int n = 500000000;
    std::vector<int> a(n);

    std::mt19937 gen(42);
    std::uniform_int_distribution<int> dist(0, 1000000000);
    for (int i = 0; i < n; i++) {
        a[i] = dist(gen);
    }

    const int NUM_RUNS = 5;
    std::vector<double> times_non_simul;
    std::vector<double> times_simul;

    int m1, M1, val_min, val_max;

    for (int r = 0; r < NUM_RUNS; r++) {
        std::cout << "Execução " << r + 1 << "\n";

        auto start_1 = std::chrono::high_resolution_clock::no
w();
        m1 = min_value(a);
        M1 = max_value(a);
        auto stop_1 = std::chrono::high_resolution_clock::now
();

        times_non_simul.push_back(std::chrono::duration<doubl
e>(stop_1 -

start_1).count());

        auto start_2 = std::chrono::high_resolution_clock::no
w();
        min_max(a, val_min, val_max);
        auto stop_2 = std::chrono::high_resolution_clock::now
();
```

```
        times_simul.push_back(std::chrono::duration<double>(s
top_2 -
                                                            s
tart_2).count());
    }

    std::cout << "Nao-simultaneo: " << get_median(times_non_s
imul) << " segundos\n";
    std::cout << "Simultaneo:     " << get_median(times_simu
l) << " segundos\n";

    std::cout << "\n(Controle - Max: " << M1 << " | Min: " <<
m1 << ")\n";

    return 0;
```

```
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/estatistica_e_ordem/minmax$ ./minmax
Execução 1
Execução 2
Execução 3
Execução 4
Execução 5
Nao-simultaneo: 4.45594 segundos
Simultaneo:     4.0211 segundos

(Controle - Max: 1000000000 | Min: 4)
```

Redução:

$$Reducao(\%) = \frac{C_{\text{não-simultâneo}} - C_{\text{simultâneo}}}{C_{\text{não-simultâneo}}} \times 100$$

$$Reducao(\%) = \frac{999.999.998 - 749.999.998}{999.999.998} \times 100 \approx \mathbf{25\%}$$

Ganho:

$$Ganho(\%) = \frac{T_{\text{não-simultâneo}} - T_{\text{simultâneo}}}{T_{\text{não-simultâneo}}} \times 100$$

$$Ganho(\%) = \frac{4.45594 - 4.0211}{4.45594} \times 100 \approx \mathbf{9{,}76\%}$$

2. **Implemente os algoritmos de seleção aleatória e seleção das seções 9.2 2 9.3 do livro do Cormen, 4a Ed., e realize experimentos**

numéricos para demonstrar em quais casos um tem vantagens com relação ao outro

```
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
#include <random>
#include <chrono>

int particiona(std::vector<int> &A, int p, int r) {
    int x = A[r];
    int i = p - 1;
    for (int j = p; j < r; ++j) {
        if (A[j] <= x) {
            ++i;
            std::swap(A[i], A[j]);
        }
    }
    std::swap(A[i + 1], A[r]);
    return i + 1;
}

int particiona_valor(std::vector<int> &A, int p, int r, int p
ivot_val) {
    int i;
    for (i = p; i < r; i++) {
        if (A[i] == pivot_val) break;
    }
    std::swap(A[i], A[r]);
    return particiona(A, p, r);
}

int particiona_aleatorizado(std::vector<int> &A, int p, int
r) {
```

```
    int i = p + rand() % (r - p + 1);
    std::swap(A[i], A[r]);
    return particiona(A, p, r);
}

int randomized_select(std::vector<int> &A, int p, int r, int
i) {
    if (p == r) return A[p];

    int q = particiona_aleatorizado(A, p, r);
    int k = q - p + 1;

    if (i == k) return A[q];
    else if (i < k) return randomized_select(A, p, q - 1, i);
    else return randomized_select(A, q + 1, r, i - k);
}

int encontra_mediana(std::vector<int> &A, int inicio, int n)
{
    std::sort(A.begin() + inicio, A.begin() + inicio + n);
    return A[inicio + n / 2];
}

int deterministic_select(std::vector<int> &A, int p, int r, i
nt i) {
    int n = r - p + 1;
    if (n <= 5) {
        std::sort(A.begin() + p, A.begin() + r + 1);
        return A[p + i - 1];
    }

    std::vector<int> medianas;
    for (int j = 0; j < n / 5; j++) {
        medianas.push_back(encontra_mediana(A, p + j * 5,
5));
    }
```

```
    if (n % 5 != 0) {
        medianas.push_back(encontra_mediana(A, p + (n / 5) *
5, n % 5));
    }

    int val_mediana_das_medianas = deterministic_select(media
nas, 0,
                                                            media
nas.size() - 1, medianas.size() / 2 + 1);

    int q = particiona_valor(A, p, r, val_mediana_das_mediana
s);
    int k = q - p + 1;

    if (i == k) return A[q];
    else if (i < k) return deterministic_select(A, p, q - 1,
i);
    else return deterministic_select(A, q + 1, r, i - k);
}

int main() {
    srand(42);
    int n = 1000000;
    int iesimo = n / 2;

    std::vector<int> v(n);
    std::iota(v.begin(), v.end(), 1);
    std::shuffle(v.begin(), v.end(), std::mt19937(42));

    std::vector<int> v_det = v;
    std::vector<int> v_rand = v;

    auto start1 = std::chrono::high_resolution_clock::now();
    int res1 = deterministic_select(v_det, 0, n - 1, iesimo);
    auto end1 = std::chrono::high_resolution_clock::now();
    double tempo_det = std::chrono::duration<double>(end1 - s
```

```
tart1).count();

    auto start2 = std::chrono::high_resolution_clock::now();
    int res2 = randomized_select(v_rand, 0, n - 1, iesimo);
    auto end2 = std::chrono::high_resolution_clock::now();
    double tempo_rand = std::chrono::duration<double>(end2 -
start2).count();

    std::cout << "Secao 9.3 (Deterministico): " << tempo_det
<< " s\n";
    std::cout << "Secao 9.2 (Aleatorio):      " << tempo_rand
<< " s\n";
    std::cout << res1 << " " << res2 << "\n";

    return 0;
}
```

Saída com os seguintes experimentos:

- Experimento 1: Vetor Aleatório.
- Experimento 2: Vetor Ordenado.
- Experimento 3: Busca do Mínimo.

```
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/estatistica_e_ordem/selecao$ ./run.sh
rm -f selecao
g++ -O3 -Wall -std=c++17 -o selecao selecao.cpp
Secao 9.3 (Deterministico): 0.0270179 s
Secao 9.2 (Aleatorio):      0.0111258 s
500000 500000
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/estatistica_e_ordem/selecao$ ./run.sh
rm -f selecao
g++ -O3 -Wall -std=c++17 -o selecao selecao.cpp
Secao 9.3 (Deterministico): 0.0234931 s
Secao 9.2 (Aleatorio):      0.00429957 s
500000 500000
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/estatistica_e_ordem/selecao$ ./run.sh
rm -f selecao
g++ -O3 -Wall -std=c++17 -o selecao selecao.cpp
Secao 9.3 (Deterministico): 0.0108978 s
Secao 9.2 (Aleatorio):      0.00268683 s
1 1
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/estatistica_e_ordem/selecao$
```

3. **Implemente o algoritmo da mediana ponderada e use-o para resolver o item e do Problema 9-3 do Cormen, 4a Ed.**

Problema:

O ***problema da localização da agência postal*** é definido da seguinte maneira: temos como entrada n pontos $p_1, p_2, ..., p_n$ com pesos associados $w_1, w_2, ..., w_n$. Uma solução é um ponto p (não necessariamente um dos pontos de entrada) que minimize o somatório $\sum_{i=1}^{n} w_i d(p, p_i)$, em que $d(a, b)$ é a distância entre os pontos a e b.

d. Mostre que a mediana ponderada é uma solução melhor para o problema da localização de agência postal unidimensional, no qual os pontos são simplesmente números reais e a distância entre os pontos a e b é $d(a, b) = |a - b|$.

e. Determine a melhor solução para o problema de localização da agência postal bidimensional, no qual os pontos são pares de coordenadas (x, y) e a distância entre os pontos $a = (x_1, y_1)$ e $b = (x_2, y_2)$ é a ***distância Manhattan*** dada por $d(a, b) = |x_1 - x_2| + |y_1 - y_2|$.

Distância Manhattan: $d(a, b) = |a_x - b_x| + |a_y - b_y|$

Função para minimizar:

$$G(x, y) = \sum_{i=1}^{n} w_i \cdot (|x - x_i| + |y - y_i|)$$

$$G(x, y) = (\sum_{i=1}^{n} w_i |x - x_i|) + (\sum_{i=1}^{n} w_i |y - y_i|)$$

```
import random

class Cidade:
    def __init__(self, id_cidade, x, y, peso):
        self.id = id_cidade
        self.x = x
        self.y = y
        self.peso = peso

def quick_select_ponderado(pontos, alvo):
    if len(pontos) == 1:
        return pontos[0][0]

    pivo = random.choice(pontos)[0]

    menores, iguais, maiores = [], [], []
    peso_menores, peso_iguais = 0, 0

    for coord, peso in pontos:
        if coord < pivo:
```

```
            menores.append((coord, peso))
            peso_menores += peso
        elif coord == pivo:
            iguais.append((coord, peso))
            peso_iguais += peso
        else:
            maiores.append((coord, peso))

    if peso_menores >= alvo:
        return quick_select_ponderado(menores, alvo)
    elif peso_menores + peso_iguais >= alvo:
        return pivo
    else:
        return quick_select_ponderado(maiores, alvo - peso_me
nores - peso_iguais)

def resolver_agencia_postal(cidades):
    print("   QUESTÃO 3: LOCALIZAÇÃO ÓTIMA (MEDIANA PONDERAD
A)")

    for c in cidades:
        print(f"Cidade {c.id}: Pos({c.x}, {c.y}) | População:
{c.peso}")

    peso_total = sum(c.peso for c in cidades)
    alvo = peso_total / 2.0

    pontos_x = [(c.x, c.peso) for c in cidades]
    pontos_y = [(c.y, c.peso) for c in cidades]

    x_otimo = quick_select_ponderado(pontos_x, alvo)
    y_otimo = quick_select_ponderado(pontos_y, alvo)

    print("-" * 55)
    print(f"Peso Total (W): {peso_total} | Alvo (W/2): {alv
o}")
```

```python
    print(f"X Ótimo: {x_otimo}")
    print(f"Y Ótimo: {y_otimo}")
    print(f"\nRESULTADO: Ponto ideal da agência em ({x_otimo}, {y_otimo})")

if __name__ == "__main__":
    lista_cidades = [
        Cidade(1, 1.0, 8.0, 450000),
        Cidade(2, 10.0, 2.0, 450000),
        Cidade(3, 5.0, 5.0, 50000)
    ]
    resolver_agencia_postal(lista_cidades)
```

saída:

```
brunopaiva@pop-os:~/programacao-paralela$ /bin/python3
Cidade 1: Pos(1.0, 8.0) | População: 450000
Cidade 2: Pos(10.0, 2.0) | População: 450000
Cidade 3: Pos(5.0, 5.0) | População: 50000
--------------------------------------------------------
Peso Total (W): 950000 | Alvo (W/2): 475000.0
X Ótimo: 5.0
Y Ótimo: 5.0

RESULTADO: Ponto ideal da agência em (5.0, 5.0)
brunopaiva@pop-os:~/programacao-paralela$
```